Time and Space Complexity

Last Updated : 31 Jul, 2025

Many times there are more than one ways to solve a problem with different algorithms and we need a way to compare multiple ways. Also, there are situations where we would like to know how much time and resources an algorithm might take when implemented. To measure performance of algorithms, we typically use time and space complexity analysis. The idea is to measure order of growths in terms of input size.

- Independent of the machine and its configuration, on which the algorithm is running on.
- Shows a direct correlation with the number of inputs.
- Can distinguish two algorithms clearly without ambiguity.

Time Complexity: The time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input. Note that the time to run is a function of the length of the input and not the actual execution time of the machine on which the algorithm is running on.

The valid algorithm takes a finite amount of time for execution. The time required by the algorithm to solve given problem is called ***time complexity*** of the algorithm. Time complexity is very useful measure in algorithm analysis.

It is the time needed for the completion of an algorithm. To estimate the time complexity, we need to consider the cost of each fundamental instruction and the number of times the instruction is executed.

Example 1: Addition of two scalar variables.

```
Algorithm ADD SCALAR(A, B)
//Description: Perform arithmetic addition of two numbers
//Input: Two scalar variables A and B
//Output: variable C, which holds the addition of A and B
C <- A + B
return C
```

The addition of two scalar numbers requires one addition operation. the time complexity of this algorithm is constant, so T(n) = O(1) .

In order to calculate time complexity on an algorithm, it is assumed that a **constant time c** is taken to execute one operation, and then the total operations for an input length on **N** are calculated. Consider an example to understand the process of calculation: Suppose a problem is to find whether a pair **(X, Y)** exists in an array, A of **N** elements whose sum is **Z**. The simplest idea is to consider every pair and check if it satisfies the given condition or not.

The pseudo-code is as follows:

```
int a[n];
for(int i = 0;i < n;i++)
  cin >> a[i]

for(int i = 0;i < n;i++)
  for(int j = 0;j < n;j++)
    if(i!=j && a[i]+a[j] == z)
       return true
return false
```

Below is the implementation of the above approach:

C++ Java Python C# JavaScript

```cpp
// C++ program for the above approach
#include <bits/stdc++.h>
using namespace std;

// Function to find a pair in the given
// array whose sum is equal to z
bool findPair(int a[], int n, int z)
{
    // Iterate through all the pairs
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)

            // Check if the sum of the pair
            // (a[i], a[j]) is equal to z
```

```
            if (i != j && a[i] + a[j] == z)
                return true;

    return false;
}

// Driver Code
int main()
{
    // Given Input
    int a[] = { 1, -2, 1, 0, 5 };
    int z = 0;
    int n = sizeof(a) / sizeof(a[0]);

    // Function Call
    if (findPair(a, n, z))
        cout << "True";
    else
        cout << "False";
    return 0;
}
```

Output

```
False
```

Assuming that each of the operations in the computer takes approximately constant time, let it be **c**. The number of lines of code executed actually depends on the value of **Z**. During analyses of the algorithm, mostly the worst-case scenario is considered, i.e., when there is no pair of elements with sum equals **Z**. In the worst case,

- **N*c** operations are required for input.
- The outer loop **i** loop runs **N** times.
- For each **i**, the inner loop **j** loop runs **N** times.

So total execution time is **N*c + N*N*c + c**. Now ignore the lower order terms since the lower order terms are relatively insignificant for large input, therefore only the highest order term is taken (without constant) which is **N*N** in this case. Different

notations are used to describe the limiting behavior of a function, but since the worst case is taken so big-O notation will be used to represent the time complexity.

Hence, the time complexity is **O(N^2)** for the above algorithm. Note that the time complexity is solely based on the number of elements in array **A** i.e the input length, so if the length of the array will increase the time of execution will also increase.

Order of growth is how the time of execution depends on the length of the input. In the above example, it is clearly evident that the time of execution quadratically depends on the length of the array. Order of growth will help to compute the running time with ease.

Another Example: Let's calculate the time complexity of the below algorithm:

C++ Java Python C# JavaScript

```
count = 0
for (int i = N; i > 0; i /= 2)
  for (int j = 0; j < i; j++)
    count++;
```

This is a tricky case. In the first look, it seems like the complexity is **O(N * log N)**. **N** for the **j's** loop and **log(N)** for **i's** loop. But it's wrong. Let's see why.

Think about how many times **count++** will run.

- When **i = N**, it will run **N** times.
- When **i = N / 2**, it will run **N / 2** times.
- When **i = N / 4**, it will run **N / 4** times.
- And so on.

The total number of times **count++** will run is **N + N/2 + N/4+...+1= 2 * N**. So the time complexity will be **O(N)**.

Some general time complexities are listed below with the input range for which they are accepted in competitive programming:

Input Length	Worst Accepted Time Complexity	Usually type of solutions
10 -12	O(N!)	Recursion and backtracking
15-18	$O(2^N * N)$	Recursion, backtracking, and bit manipulation
18-22	$O(2^N * N)$	Recursion, backtracking, and bit manipulation
30-40	$O(2^{N/2} * N)$	Meet in the middle, Divide and Conquer
100	$O(N^4)$	Dynamic programming, Constructive
400	$O(N^3)$	Dynamic programming, Constructive
2K	$O(N^2 * \log N)$	Dynamic programming, Binary Search, Sorting, Divide and Conquer
10K	$O(N^2)$	Dynamic programming, Graph, Trees, Constructive
1M	O(N* log N)	Sorting, Binary Search, Divide and Conquer
100M	O(N), O(log N), O(1)	Constructive, Mathematical, Greedy Algorithms

Space Complexity:

Definition -

Problem-solving using computer requires memory to hold temporary data or final result while the program is in execution. The amount of memory required by the algorithm to solve given problem is called **space complexity** of the algorithm.

The space complexity of an algorithm quantifies the amount of space taken by an algorithm to run as a function of the length of the input. Consider an example: Suppose a problem to find the frequency of array elements.

It is the amount of memory needed for the completion of an algorithm.

To estimate the memory requirement we need to focus on two parts:

(1) A fixed part: It is independent of the input size. It includes memory for instructions (code), constants, variables, etc.

(2) A variable part: It is dependent on the input size. It includes memory for recursion stack, referenced variables, etc.

Example : Addition of two scalar variables

```
Algorithm ADD SCALAR(A, B)
//Description: Perform arithmetic addition of two numbers
//Input: Two scalar variables A and B
//Output: variable C, which holds the addition of A and B
C <– A+B
return C
```

The addition of two scalar numbers requires one extra memory location to hold the result. Thus the space complexity of this algorithm is constant, hence S(n) = O(1).

The pseudo-code is as follows:

```
int freq[n];
int a[n];
for(int i = 0; i<n; i++)
{
   cin>>a[i];
   freq[a[i]]++;
}
```

Below is the implementation of the above approach:

C++ Java Python C# JavaScript

```cpp
// C++ program for the above approach
#include <bits/stdc++.h>
using namespace std;

// Function to count frequencies of array items
void countFreq(int arr[], int n)
{
    unordered_map<int, int> freq;

    // Traverse through array elements and
    // count frequencies
    for (int i = 0; i < n; i++)
        freq[arr[i]]++;

    // Traverse through map and print frequencies
    for (auto x : freq)
        cout << x.first << " " << x.second << endl;
}

// Driver Code
int main()
{
    // Given array
    int arr[] = { 10, 20, 20, 10, 10, 20, 5, 20 };
    int n = sizeof(arr) / sizeof(arr[0]);

    // Function Call
    countFreq(arr, n);
    return 0;
}
```

Output

```
5 1
20 4
10 3
```

Here two arrays of length **N**, and variable **i** are used in the algorithm so, the total space used is **N * c + N * c + 1 * c = 2N * c + c**, where **c** is a unit space taken. For

many inputs, constant **c** is insignificant, and it can be said that the space complexity is **O(N)**.

There is also **auxiliary space,** which is different from space complexity. The main difference is where space complexity quantifies the total space used by the algorithm, auxiliary space quantifies the extra space that is used in the algorithm apart from the given input. In the above example, the auxiliary space is the space used by the freq[] array because that is not part of the given input. So total auxiliary space is **N * c + c** which is **O(N)** only.